

Ocar: Auto Rickshaw In-City Rental

Feature architecture: a new vehicle category, its package pricing, and the platform changes required to support it

Version 1.0 | September 2026

This document describes what is being added to the platform to support auto rickshaws as a bookable in-city vehicle category, priced by fixed package with a minimum fare and distance/time overage, and every layer of the system that changes to make that work: the pricing data model, the backend booking and fare-settlement logic, and the three frontend applications.

1. Overview	3
2. Data layer changes	
3. Backend changes	
4. Frontend changes	
5. Ride lifecycle, booking through settlement	
6. Components touched	

1. Overview

Auto rickshaw is added as a new vehicle category, alongside the existing hatchback, sedan, SUV, luxury, and van categories. It is scoped to in-city travel only, and is booked exclusively as a rental: the rider picks a fixed package (a bundled amount of time and distance for one price) rather than paying a per-kilometre one-way fare. Distance or time driven beyond the package's allowance is billed at that package's own overage rate when the trip ends.

The category launches with multiple package tiers, and is treated as fully isolated from every other category: an auto rickshaw driver is never offered a sedan booking as a fallback, and vice versa, since the two are not interchangeable for a rider's purpose. Supporting this required changes across four areas: the pricing and category data model, the fare-estimation and booking logic, the trip-completion and fare-settlement logic, and the rider, admin, and driver-facing screens that display and configure it.

Pricing follows the same package structure already used by the existing rental categories for both in-city and outstation bookings: a duration, a distance allowance, a package fare, and two overage rates, one per kilometre and one per minute, charged only on the amount driven past the allowance. Auto rickshaw launches with multiple selectable tiers built on this structure rather than a single fixed package, and the exact tiers and rates are configured separately.

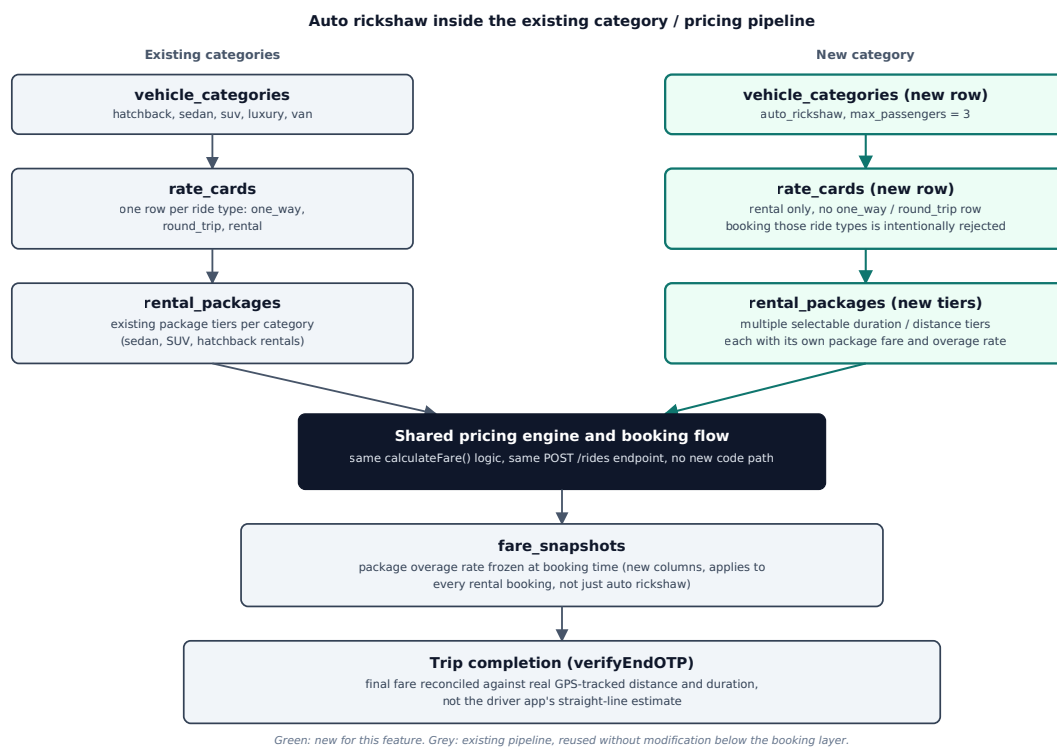
2. Data layer changes

Two migrations support this feature. The first introduces the category itself and its pricing:

Table	Change
<code>vehicle_categories</code>	One new row: <code>auto_rickshaw</code> , maximum 3 passengers.
<code>rate_cards</code>	One new row, for the <code>rental</code> ride type only. No <code>one_way</code> or <code>round_trip</code> row is created for this category; booking those ride types against it is rejected by design, since auto rickshaw is package-priced in-city travel only.
<code>stop_charges</code>	One new row, required by the schema for every category regardless of whether multi-stop rentals are commonly used for this vehicle type.
<code>rental_packages</code>	Multiple new rows, one per launch package tier for this category, each bundling a time and distance allowance for a fixed fare with its own overage rate.
<code>vehicle_brands</code> / <code>vehicle_models</code>	Five manufacturers (Bajaj, Piaggio, Atul, TVS, Mahindra) and their full model lineups seeded, so drivers registering an auto rickshaw during onboarding have real options in the make and model dropdown rather than an empty list. Fuel type (petrol, diesel, CNG, electric) is already collected per vehicle at registration regardless of category, so no separate work was needed there.
<code>category_fallback_rules</code>	

No rows added, in either direction. This is what makes the category fully isolated: the ride-assignment logic never substitutes an auto rickshaw for another category's booking or the reverse.

The second migration adds four nullable columns to `fare_snapshots`: the package's overage rate per kilometre, its overage rate per minute, its distance allowance, and its time allowance. This is necessary because, unlike rate cards, rental packages are not versioned; an admin edit to a package's pricing updates that package's row directly rather than creating a new one. Without freezing the terms a rider was quoted onto their own ride record at booking time, a package price change made while a trip is already in progress could silently alter what that rider is charged at settlement. This applies to every rental booking, not only auto rickshaw, since the same package model is shared across categories.



Auto rickshaw's category, rate card, and packages sit alongside the existing ones, converging on the same shared pricing engine, booking flow, and settlement logic below the category layer.

3. Backend changes

3.1 Fare estimation

The fare-estimate endpoint, already called whenever a rider views package options before booking, is extended to return the selected package's overage rates and allowance alongside its price, so those terms can be captured at booking time in the next step.

3.2 Booking

When a rental ride is created, the returned package terms are written onto that ride's fare record. This is the freeze described in Section 2: from this point forward, the ride's own record,

not the live `rental_packages` row, is the source of truth for what this specific rider agreed to pay for overage.

3.3 Trip completion and fare settlement

At the end of a trip, the platform already reconstructs the actual distance driven from the GPS location pings recorded throughout the ride, rather than trusting the driver app's own on-device estimate, for outstation round trips. This same reconciliation is extended to cover rental rides. Concretely: the billed distance and duration for overage purposes now come from the recorded GPS trail whenever enough location data was captured during the trip, with the same safeguard already in place elsewhere, a sanity ceiling that discards a GPS trail if it implies an implausible distance, falling back instead to the client-reported estimate and flagging the ride for review. This closes a gap that existed for every rental category before this feature, where final billing relied on an estimate rather than measured movement, and is not specific to auto rickshaw.

Overage is then computed against the package's frozen allowance from Section 2, at its frozen overage rate, and added to the package fare to produce the final amount shown to the rider.

4. Frontend changes

4.1 Rider app

The in-city rental screen is being fully redesigned, not just extended to list a new option. The rebuild covers how vehicle categories and their package tiers are presented and compared, laid out to current UX conventions for package selection so the added category reads as a natural part of the screen rather than an item bolted onto an older layout, and so the full set of categories, auto rickshaw included, stays easy to scan and select on a phone screen.

As part of that redesign, a new vehicle silhouette was added for auto rickshaw's three-wheeler body shape, so the category is visually distinct from the four-wheeled options rather than falling back to a generic car icon.

A related fix was required on the separate one-way and round-trip booking screen, which is not part of this redesign. That screen renders every available vehicle category and fetches a live fare for each one; every category up to now has supported all three ride types, so this was never an issue. Auto rickshaw is the first category that does not support one-way or round-trip travel, and without a fix it would have appeared there as a selectable card with no valid fare behind it. The screen now only displays categories for which a fare was actually returned.

4.2 Admin portal

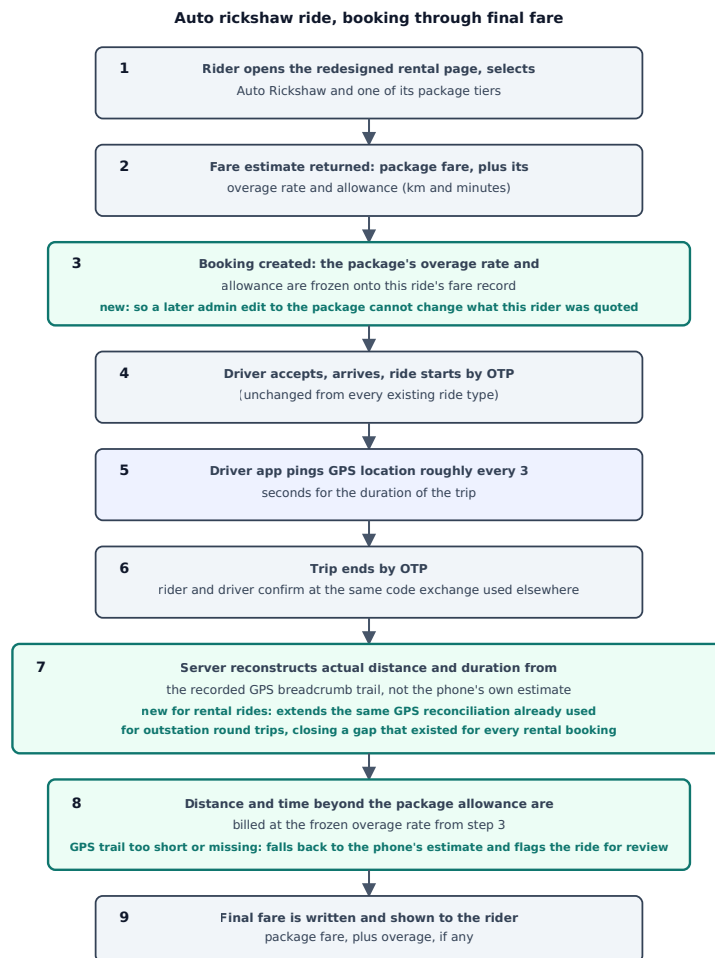
Auto rickshaw was added to the category ordering used by the rate-card configuration screen and its rental-packages tab, so its pricing and package tiers are visible and editable there. No new admin screen was built; the existing rate-card and package management interface, already generic across categories, handles this category the same way it handles the others.

4.3 Driver app

No code change was required. The vehicle-category selector shown during driver onboarding is already populated from the categories list returned by the backend, so auto rickshaw and its

seeded brand and model options appear there automatically once the category exists in the data layer.

5. Ride lifecycle, booking through settlement



Steps in grey are the existing rental ride flow, unchanged. Steps in green are new work introduced for this feature, applied to every rental ride, not only auto rickshaw.

The booking, ride, and OTP flow is identical to every other rental ride. The new work sits at the two points marked in green: freezing package terms at booking, and reconciling the final fare against measured GPS distance rather than an on-device estimate.

6. Components touched

Layer	Change
Database	Two migrations: category, rate card, stop charges, package tiers, and brand/model seed data; and four new columns on the fare-snapshot table.
Pricing service	Fare-estimate response extended to carry a package's overage terms.
Rides service	

	Booking persists the frozen package terms; trip completion gains a rental-specific reconciliation branch against GPS-tracked distance and duration, with its own fallback and review-flagging path.
Rider app	Rental screen redesigned to present categories and package tiers, auto rickshaw included; new vehicle icon; category-list fix on the one-way/round-trip screen.
Admin portal	Category ordering updated in the rate-card screen and its rental-packages tab.
Driver app	No change; onboarding is data-driven from the categories already added in the database layer.
Test coverage	New unit tests for the rental trip-completion reconciliation branch, covering both the GPS-tracked path and the fallback path.